COMPUTER SCIENCE + COMPUTER ENGINEERING

A guided undergraduate-to-graduate course for games

03

Module 09: Probability, statistics and evidence

Expanded Volume 02 / individual teaching module

Four lessons with derivations, runnable examples, practical work and 20 quiz questions.

PACKAGE 03 / EXPANDED VOLUME 02 / SEPTEMBER 28, 2026

Self-study: complete the derivations, implementation, testing and review.

Module 09 — Probability, statistics and evidence

Original prerequisites: Modules 06 and 08.2.

Original module project: Create a reproducible benchmark experiment and a loot-distribution simulation. Report assumptions, uncertainty, representative workloads and one result that did not support the original hypothesis.

Expanded project: Randomness and performance evidence. A detector contingency table, reward-distribution experiment with recorded seeds, and one locally measured comparison with randomized paired ordering, raw results, descriptive summaries, qualified uncertainty, and a held-out prediction exercise.

Acceptance: Synthetic and measured datasets remain separately labeled. Both algorithms produce equivalent answers. Report workload, machine/software boundary, warmup, trials, inclusion rules, percentile convention, and independence limitations. A null or adverse result is acceptable.

This module has four lessons, 20 lesson quiz questions, 15 written problems and five review questions. The written problems and integration project are in the workbook.

Lesson 09.1 — Probability and conditional reasoning

Learning outcome: Model uncertainty without confusing independent and conditional events.

Define the sample space and event

A probability model assigns nonnegative weights summing to one over possible outcomes. An event is a set of outcomes. For two independent fair coin flips, the equally likely outcomes are HH, HT, TH, TT. The event "two heads" contains one of them, so its probability is 1/4.

If the second flip is instead copied from the first, the model has only HH and TT, each with probability 1/2. Both individual positions still have a 1/2 chance of heads, but the joint probability changes. Matching marginal probabilities does not establish independence.

Conditioning changes the population

Conditional probability P(A|B)=P(A intersection B)/P(B) requires P(B)>0. The denominator restricts attention to outcomes where B occurred. It is not generally symmetric: P(alert|bug) considers buggy cases; P(bug|alert) considers alerted cases.

Events A and B are independent when P(A intersection B)=P(A)P(B). When P(B)>0, this is equivalent to P(A|B)=P(A). Do not multiply probabilities merely because observations occur at different times. Game systems can share state, random streams, players, or workload phases that create dependence.

The complement rule P(not A)=1-P(A) often simplifies calculations. For independent attempts with success probability p, the probability of at least one success in n attempts is 1-(1-p)^n. The independence and fixed-p assumptions are essential; a pity mechanic or cooldown changes the model.

Reverse a conditional with Bayes' rule

For a detector, let p be the prior bug probability, s its sensitivity P(alert|bug), and f its false-positive probability P(alert|no bug). The total alert probability is p*s+(1-p)*f. Bayes' rule gives P(bug|alert)=p*s/[p*s+(1-p)*f].

A high sensitivity does not mean most alerts are true. When true bugs are rare, even a modest false-positive rate can contribute many more alerts from nonbug cases. A contingency table makes the changing denominators visible.

If the total alert probability is zero, the conditional on alert is undefined in this model. Returning zero would pretend a meaningful probability was computed. The reference rejects that situation explicitly.

Simulations check implementations, not assumptions

A seeded simulation can test whether the sampling code approaches a known model probability across many trials. It does not prove that a real detector or real players follow that model. Use synthetic detector values here; they are chosen learning parameters, not measured product claims.

A frequency may differ from the model probability because of ordinary sampling variation, a biased sampler, dependence, or a coding defect. Keep those explanations separate. First validate a small

finite sample space by enumeration. Then compare a larger simulation and document the seed, generator, trial count, and event definitions.

For an engineering report, state the population and time window to which an estimate applies. A test suite's failure rate on deliberately adversarial inputs is not automatically the failure rate across ordinary players. Selection of inputs is part of the probability model, not a minor reporting detail. [R09, R06]

Worked derivation

Imagine 10,000 synthetic test cases. Exactly 100 contain a bug; 9,900 do not. At sensitivity 90%, 90 buggy cases alert and 10 do not. At false-positive rate 5%, 495 nonbug cases alert and 9,405 do not.

There are 585 alerts, only 90 of which are true bugs. Thus P(bug|alert)=90/585=2/13, approximately 0.153846. In contrast, P(alert|bug)=90/100=0.9. Both are correct; they use different denominators.

For two independent attempts with success probability 0.2, the probability of at least one success is 1-0.8^2=0.36. Copying the first outcome to the second would instead leave that probability at 0.2.

Runnable demonstration

Run `python run_example.py 09.1` from the extracted package folder. Predict before executing.

```
from mathkit import posterior_bug
bug_count = 100
nonbug_count = 9900
true_alerts = int(bug_count * 0.9)
false_alerts = int(nonbug_count * 0.05)
print("true and false alerts:", true_alerts, false_alerts)
print(f"P(bug|alert): {posterior_bug(0.01, 0.9, 0.05):.6f}")
print("independent two heads:", 0.5 * 0.5)
print("copied second coin, two heads:", 0.5)
```

Recorded expected output:

```
true and false alerts: 90 495
P(bug|alert): 0.153846
independent two heads: 0.25
copied second coin, two heads: 0.5
```

Practical assignment

Original brief: Simulate independent and dependent binary events. Build a small contingency table for a hypothetical bug detector. Pass: calculate both P(alert|bug) and P(bug|alert) and explain why they differ.

1. Implement posterior_bug(prior,sensitivity,false_positive), validating probability ranges and nonzero alert probability.
2. Construct the full four-cell detector table, including true/false positives and negatives, before using the formula.
3. Simulate independent binary events and a copied-outcome process with the same marginal rates; compare their joint events.

4. Vary only the base rate over 0.001,0.01,0.1 and explain the different posterior probabilities without calling one detector universally accurate.

Coding slice: Work in `labs/student/lab_09_1.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

An alert rate, sensitivity, false-positive rate, and positive predictive probability are different quantities. Do not swap their denominators or assume independence from temporal separation.

Independent transfer task

Keep sensitivity 0.9 and false-positive rate 0.05, but change the prior bug probability to 0.1. Find P(bug|alert).

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the probability of two independent events with probabilities 0.5 and 0.2 both occurring?
2. Is P(A|B) generally equal to P(B|A)?
3. Why do rare-event base rates matter for an alert?
4. What is the probability of at least one success in three independent trials with p=0.5?
5. When is P(bug|alert) undefined in the detector model?

Assigned study

R09: conditional probability, Bayes theorem, independence, and total probability. R06: finite sample spaces and event counting.

Original reading assignment retained: R09: probability, conditional probability and Bayes' theorem.

Lesson 09.2 — Random variables, distributions and sampling

Learning outcome: Distinguish an expected value from a guaranteed outcome.

A random variable is a numerical outcome

Let reward X be 0, 5, or 20 with probabilities 0.70, 0.25, and 0.05. Its expectation is E[X]=sum(p_i*x_i). It describes an average under the model, not an item that must appear in a single trial. The expected value 2.25 is not even an allowed individual reward here.

Variance is E[(X-E[X])^2], equivalently E[X^2]-E[X]^2. Standard deviation is its square root and has the original units. A mean without a spread can hide very different experiences: a constant reward and a rare large reward can have the same expectation but different variance.

Linearity of expectation does not require independence: E[X+Y]=E[X]+E[Y] whenever the expectations are defined. Adding variances does require care; Var(X+Y)=Var(X)+Var(Y)+2*Cov(X,Y). Independence makes the covariance zero, but simply sharing a mean does not.

Different distributions encode different assumptions

A Bernoulli variable describes one binary trial. A binomial count describes the number of successes in n independent Bernoulli trials with the same probability p; it has mean np and variance np(1-p). A normal distribution is continuous and can approximate some aggregated measurements, but it should not be chosen just because a histogram looks vaguely bell-shaped.

Repeated rewards with a changing probability, a without-replacement draw, or a shared random event do not automatically form a binomial model. Start with the mechanism, then choose the distribution.

A weighted sampler partitions a unit interval

For nonnegative weights, divide by their positive total to obtain probabilities. Cumulative boundaries partition u in [0,1) into intervals. With weights 70,25,5, the intervals are [0,0.70), [0.70,0.95), and [0.95,1). The boundary rule must be explicit: exactly 0.70 belongs to the second interval.

Zero-weight categories have empty intervals and should never be selected. Negative weights, a zero total, and u outside [0,1) are invalid. This bundle separates deterministic weighted_index(weights,u) from the generator producing u. That makes edge cases testable without hoping a random run happens to hit a boundary.

Reproducibility is controlled state

A pseudorandom generator is deterministic given its state and algorithm. A seed helps replay an experiment when the generator and call sequence are controlled. Adding an unrelated random draw to one shared stream can shift every later outcome. Separate streams for unrelated subsystems when that independence of implementation changes is a requirement.

Python's random module is used here for learning simulations, not security-sensitive decisions. Record the interpreter and generator along with the seed; do not promise identical behavior across arbitrary future tool versions or changes to sampling APIs. [R04]

Interpret frequencies at the right scale

For a category probability p and n independent draws, the standard deviation of the observed proportion is sqrt(p(1-p)/n). This is a model calculation, not a guaranteed error bound. Multiplying n by four reduces that standard deviation by two, not four.

A single seeded run is an illustration. Compare several sample sizes and seeds while preserving the planned analysis. Do not rerun until the numbers look impressively close to the expectation and then report only that run. [R09]

Worked derivation

For rewards (0,5,20) and probabilities (0.70,0.25,0.05), the expectation is 0+1.25+1=2.25. E[X^2]=0+25*0.25+400*0.05=26.25, so variance=26.25-2.25^2=21.1875.

The demonstration's 1,000 draws with seed 2026 produce counts [725,220,55] in the tested environment. The observed mean is (220*5+55*20)/1000=2.2. It differs from 2.25 without necessarily indicating a bug.

For the rare category p=0.05 and n=1000, the expected count is 50 and the count standard deviation is sqrt(1000*0.05*0.95), approximately 6.892. The observed count 55 is an observation, not proof that the true probability is 0.055.

Runnable demonstration

Run `python run_example.py 09.2` from the extracted package folder. Predict before executing.

```
import random
from mathkit import weighted_index
rng = random.Random(2026)
weights, rewards = (70, 25, 5), (0, 5, 20)
counts = [0, 0, 0]
for _ in range(1000):
    counts[weighted_index(weights, rng.random())] += 1
print("seed: 2026; trials: 1000")
print("counts:", counts)
expected = sum(w*r for w, r in zip(weights, rewards))/sum(weights)
print(f"model expectation: {expected:.3f}")
print(f"sample mean: {sum(c*r for c,r in zip(counts,rewards))/1000:.3f}")
```

Recorded expected output:

```
seed: 2026; trials: 1000
counts: [725, 220, 55]
model expectation: 2.250
sample mean: 2.200
```

Practical assignment

Original brief: Implement a weighted reward sampler and compare observed frequencies to the intended distribution over several sample sizes. Pass: record seed, generator, counts and expected values; explain finite-sample variation.

1. Implement weighted_index(weights,u), keeping the random generator outside this deterministic function. Test boundaries, zero weights, invalid u, and invalid totals.

2. Simulate the three-category reward model at 100,1000,10000 draws and at least three recorded seeds.
3. Derive the expectation and variance by hand; compare empirical counts and means without demanding exact frequency matches.
4. Demonstrate how an extra draw changes a shared stream, then show two separate generator instances preserve the unrelated subsystem sequence.

Coding slice: Work in `labs/student/lab_09_2.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

An expected value is not a per-trial guarantee. Setting a seed does not establish statistical independence, unbiased sampling, or security.

Independent transfer task

If two rewards are perfectly copied from one another, each with variance s^2, what is the variance of their sum?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. What is the expectation of 20 with probability 0.25 and 0 otherwise?
2. Does an expectation of 5 guarantee a reward of 5 on a trial?
3. Why separate random streams for unrelated systems?
4. Under the specified weighted intervals, which category contains u=0.70 for weights (70,25,5)?
5. Does linearity of expectation require independent variables?

Assigned study

R09: random variables, expectation, variance, Bernoulli/binomial models, and sampling. R04: random.Random, seeding, and reproducibility limitations.

Original reading assignment retained: R09: random variables, expectation and variance; R04: random reference.

Lesson 09.3 — Inference, uncertainty and benchmark design

Learning outcome: Compare implementations without hiding variance or bias.

Name the estimand and the experimental unit

An estimand is the quantity you intend to learn: mean update time per independent run, p99 frame latency for a workload, or the mean paired difference between implementations. These are not interchangeable. Decide which one matters before collecting data.

A frame inside a long run is often correlated with nearby frames through workload phases, caches, scheduling, or temperature. Counting every frame as an independent experiment can make uncertainty look much smaller than justified. A run or a randomized paired block may be the more defensible experimental unit, but independence is still an assumption to assess.

Summaries reveal different behavior

The mean adds values and divides by their count. The median gives a central order statistic and is less affected by a few large values. A tail percentile asks where a specified fraction of observed values lies. Different interpolation conventions can produce different sample percentiles, so this bundle defines the nearest-rank rule explicitly.

For n sorted observations and p in (0,1], choose the element at one-based rank ceil(p*n). For p=0, use the minimum. With only twenty samples, p99 is simply the maximum; it is not a precise estimate of a population tail. Always report sample size and the convention.

Pair comparable runs

To compare A and B, use the same declared workload and calculate d_i=B_i-A_i within each pair. Positive differences mean B took longer. Randomize whether A or B runs first within each pair to reduce order-related confounding. Warm up both, exclude only according to a predefined rule, verify equivalent results, and record every raw pair.

Separate per-operation time from whole-frame time. A faster helper function does not establish a faster game if it is a tiny fraction of the frame or changes the work performed. State whether allocation, setup, validation, rendering, and I/O are included.

Confidence intervals are conditional procedures

For independent normally distributed paired differences with unknown variance, a Student t interval for their mean uses mean(d) plus or minus t_critical*s_d/sqrt(n), with n-1 degrees of freedom. The reference mean_interval routine accepts the critical value explicitly; it does not infer a distribution, select a test, or verify independence.

The 95% interpretation concerns coverage across repeated datasets generated under the model. A realized frequentist interval is not a 95% probability statement about a fixed unknown parameter. Model violations, biased input selection, and cherry-picked stopping can invalidate the nominal coverage.

For five differences, the illustrative two-sided 95% t critical value is approximately 2.776445. This is a small teaching case; it is not a recommendation to use five runs for every performance study.

Statistical evidence is not practical importance

A p-value measures how unusual a statistic at least as extreme as observed would be under a specified null model. It is not the probability the null hypothesis is true, nor a measure of effect size. A tiny statistically detectable improvement can be operationally irrelevant; a rare visible pause can matter despite a small change in the mean.

Report differences, uncertainty assumptions, tails, and practical requirements. Preserve negative results and do not stop only when the result crosses a preferred threshold. [R09]

Worked derivation

Consider the synthetic frame list containing nineteen values of 5 ms and one value of 100 ms. Its mean is (95+100)/20=9.75 ms. Under nearest-rank percentiles, p95 has rank 19 and equals 5 ms; p99 has rank 20 and equals 100 ms. The outlier is real within the synthetic example, but p95 alone misses it.

For illustrative paired differences [1,2,3,4,5] ms, the mean is 3 ms. The sample variance is 10/4=2.5 ms^2, and the standard error is sqrt(2.5/5)=sqrt(0.5) ms. With critical value 2.776445, the interval is approximately [1.037,4.963] ms, under the stated independent-normal model.

Runnable demonstration

Run `python run_example.py 09.3` from the extracted package folder. Predict before executing.

```
from mathkit import percentile, mean_interval
frames = [5.0]*19 + [100.0]  # synthetic, not a measured machine
print(f"synthetic mean: {sum(frames)/len(frames):.3f} ms")
print("nearest-rank p95 and p99:", percentile(frames, 0.95), percentile(frames, 0.99))
# Five illustrative independent normal run differences; B minus A in ms.
mean, low, high = mean_interval([1, 2, 3, 4, 5], 2.776445)
print(f"paired mean and illustrative t interval: {mean:.3f} [{low:.3f}, {high:.3f}]")
```

Recorded expected output:

```
synthetic mean: 9.750 ms
nearest-rank p95 and p99: 5.0 100.0
paired mean and illustrative t interval: 3.000 [1.037, 4.963]
```

Practical assignment

Original brief: Compare two implementations using the same inputs, randomized test order and repeated runs. Pass: report median, a tail percentile, sample count and limitations; distinguish observations from causal claims.

1. Implement percentile(values,p) with the nearest-rank convention, and mean_interval(values,critical) using sample variance with denominator n-1.
2. Run the included benchmark runner or two independently implemented equivalent operations. Preserve randomized pair order, warmup, raw samples, and the workload.

3. Report mean, median, p95 or p99 with their convention and sample count, plus paired differences and clearly qualified uncertainty.
4. Analyze the synthetic tail example separately. Never relabel its chosen numbers as measured hardware performance.

Coding slice: Work in `labs/student/lab_09_3.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Do not treat correlated frames as independent trials, label a sample maximum a well-estimated p99, or interpret statistical significance as a useful speedup.

Independent transfer task

A is 5 ms and B is 4 ms in each paired run. What sign does B-A have, and what does that sign mean?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why can adjacent frame times violate an independence assumption?
2. Does a small p-value establish a large useful improvement?
3. Why report a tail percentile alongside a mean?
4. For nineteen 5 ms samples and one 100 ms sample, what is nearest-rank p95?
5. What does a p-value not tell you about an optimization?

Assigned study

R09: confidence intervals, Student t models, hypothesis tests, and experimental reasoning. The supplied five-point t example states its distributional assumptions explicitly.

Original reading assignment retained: R09: confidence intervals, hypothesis testing and experimental reasoning.

Lesson 09.4 — Regression, correlation and ethical telemetry

Learning outcome: Fit a model while recognizing confounding and data limits.

A fitted relationship has a scope

A regression model predicts a response from specified inputs. For update time versus entity count, a simple model is time approximately a*count+b. The intercept may represent fixed overhead and the slope incremental work, but those interpretations require a controlled experiment and an adequate model. A fitted coefficient is not automatically a physical constant.

Least squares minimizes the squared residuals on the fitting data. It does not guarantee good prediction on new workload families, different machines, or larger counts outside the tested range. Cache transitions, changing algorithms, and scheduling can make the relationship nonlinear or piecewise.

Separate fitting from evaluation

Choose training and held-out sets before examining the desired final metric. Fit a and b on training observations only, then evaluate predictions against the held-out responses. Changing the model repeatedly based on the held-out set turns it into tuning data; a final untouched test set or a clearly disclosed validation strategy is then needed.

For correlated observations, split at an appropriate group level. Putting adjacent frames from the same run into both training and test sets can leak workload-specific information and exaggerate generalization. Holding out entire runs, scenes, or count regimes can answer a more meaningful transfer question.

Define and interpret prediction error

A residual is observed minus predicted. Mean squared error averages squared residuals; root mean squared error (RMSE) takes the square root and returns to the response's units. Mean absolute error measures a different objective and is less dominated by a single large residual. A good report includes residual patterns, not only one aggregate number.

Compare with a simple baseline, such as predicting the training-set mean for every held-out observation. A complicated model should earn its complexity with relevant evidence. Report both interpolation within the training range and extrapolation outside it as distinct cases.

Correlation is not a causal explanation

Two variables can move together because of a shared cause. Scene complexity might raise both entity count and GPU time. Hardware class might influence both settings choices and measured performance. A regression using only one observed predictor can incorrectly attribute the whole association to that predictor.

A controlled benchmark that deliberately changes entity count while holding other relevant factors fixed gives stronger evidence for that intervention within the test conditions. It still does not establish

every real game's behavior. Randomization helps with some confounders, but does not repair a measurement that omits the main work or compares incorrect implementations.

Collect only data needed for the question

The supplied datasets are synthetic or local benchmark outputs. They contain no real player identifiers. For a course experiment, entity count, trial ID, algorithm, duration, environment version, and correctness status are usually enough. Do not add names, account identifiers, precise locations, chat logs, or unrelated activity simply because a telemetry system could collect them.

Write a short data plan stating purpose, fields, retention, who can read the files, and how results will be shared. A filename or machine identifier can itself expose unnecessary information; the runner avoids collecting a username or hostname. Professional measurement includes provenance and appropriate boundaries, not only statistical formulas. [R09; course data-minimization policy]

Worked derivation

Use synthetic training pairs x=(1,2,3,4), y=(3,5,7,9). They lie exactly on y=2x+1, so the training residuals are zero. Hold out x=(5,6), with observed y=(11,14). Predictions are (11,13), so residuals are (0,1).

The held-out RMSE is sqrt((0^2+1^2)/2)=sqrt(0.5), approximately 0.707107 response units. Zero training error did not imply zero held-out error.

The training-mean baseline predicts 6 for each held-out point. Its RMSE is sqrt(((11-6)^2+(14-6)^2)/2)=sqrt(44.5), approximately 6.671. The line is better on this chosen test, but the small synthetic dataset does not establish a universal performance law.

Runnable demonstration

Run `python run_example.py 09.4` from the extracted package folder. Predict before executing.

```
from mathkit import fit_line, rmse
train_x, train_y = (1, 2, 3, 4), (3, 5, 7, 9)
slope, intercept = fit_line(train_x, train_y)
test_x, test_y = (5, 6), (11, 14)  # synthetic held-out observations
predicted = [slope*x + intercept for x in test_x]
print("fitted slope and intercept:", slope, intercept)
print("held-out predictions:", predicted)
print(f"held-out RMSE: {rmse(test_y, predicted):.6f}")
```

Recorded expected output:

```
fitted slope and intercept: 2.0 1.0
held-out predictions: [11.0, 13.0]
held-out RMSE: 0.707107
```

Practical assignment

Original brief: Fit a simple performance model using entity count and measured update time. Hold out some workloads. Pass: report prediction error and one confounder; write a short data-minimization plan for the experiment.

1. Implement rmse(actual,predicted) for nonempty equal-length finite sequences. Reuse the independently tested line-fitting routine.

2. Fit update time versus entity count from declared local measurements; reserve at least one workload group before fitting. The included synthetic CSV is practice only.
3. Compare the fitted model to a training-mean baseline, preserve residuals, and distinguish interpolation from extrapolation.
4. Write a data-minimization and provenance note, identifying one possible confounder and which claims the experiment does not support.

Coding slice: Work in `labs/student/lab_09_4.py` and use the contract in the workbook. Passing that checker does not complete the written/experimental portions.

Common failure to catch

Perfect training fit is not independent evaluation. A correlated predictor is not necessarily the cause, and synthetic timings must not be reported as observed machine results.

Independent transfer task

Why is randomly splitting adjacent frames from one run into training and test sets potentially misleading?

Lesson quiz — five questions

Questions 1-3 are retained core questions; questions 4-5 apply the ideas to new cases. Answer before checking the separate key.

1. Why is training-set fit alone insufficient?
2. What is a confounder?
3. Why should telemetry collect only what the question requires?
4. Predictions are [2,4] and observations [3,4]. What is RMSE?
5. What simple baseline should accompany a fitted performance model?

Assigned study

R09: regression, inference, and model evaluation. The privacy/data-minimization requirements are course design rules, not a legal-compliance certification.

Original reading assignment retained: R09: regression and inference; R01: ethics and data-management breadth.